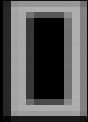


Pong Robot

Boschetti Zakaria
Bourzami Abdelmoumen



Pong est un des premiers jeux vidéo d'arcade et le premier jeu vidéo d'arcade de sport.



Le concept original de *Pong* est une simulation simpliste de tennis de table



Résumé du parallèle Pong ↔ Robot



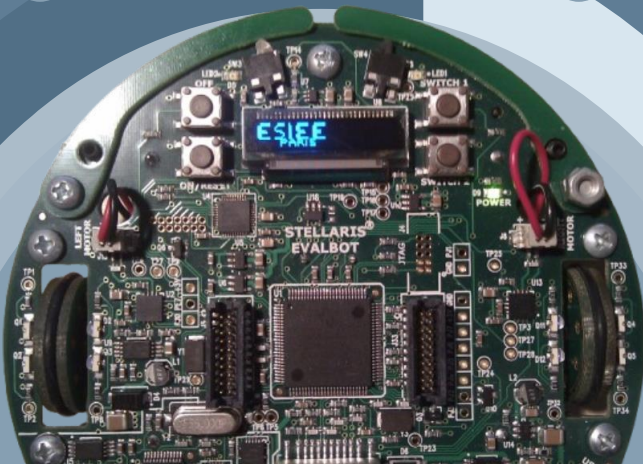
LED : représente le score du joueur ; elle clignote autant de fois que de points marqués, comme l'affichage du score en haut de l'écran dans Pong.

Bumper : détecte un choc ce qui fait rebondir le robot, comme si elle touchait une paroi du terrain Pong.



Moteur : contrôle le mouvement du robot pour lui permettre de rebondir et simuler le déplacement de la balle sur le terrain.

Switch : permet de modifier le score des joueurs; lorsqu'il marque un point, imitant l'action lorsque la balle franchit le terrain de la cage.





Fonctionnalités principales



01

Déplacement

Le robot avance en ligne droite grâce aux deux moteurs activés en continu.



02

Détection de collision

Les bumpers avant gauche et droit détectent un contact avec un obstacle (mur du terrain).



03

Réaction

Si bumper gauche toucher
→ rotation droite d'une durée aléatoire

Si bumper droit toucher
→ rotation gauche d'une durée aléatoire

Si les deux bumpers touchent
→ demi-tour complet



04

Score manuel

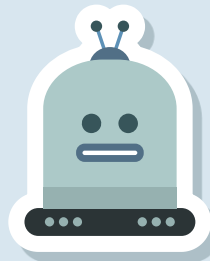
Deux boutons SW1 (joueur 1) et SW2 (joueur 2) permettent d'incrémenter le score.



05

Affichage du score

LEDs clignotent selon la valeur du score (0 → 5 puis retour à 0).



Exemple du code

Bumper gauche touché

```
178      BL      MOTEUR_GAUCHE_ARRIERE
179      BL      MOTEUR_DROIT_AVANT
180      BL      RANDOM_DELAY
```

Effet : le robot **pivote** (gauche en arrière, droite en avant) → rotation **vers la droite** (sens horaire) pendant une **durée aléatoire**

Lecture des boutons

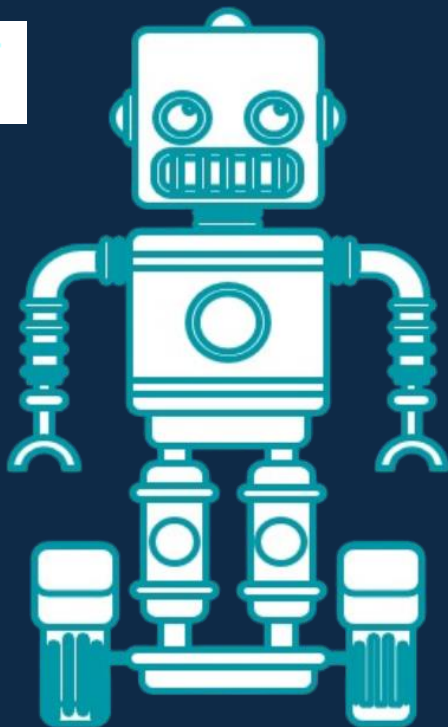
```
132      TST      r9, #BROCHE_SW1
133      BNE      NoPress1
134      ADD      r4, r4, #1
135      CMP      r4, #6
136      BLT      Keep1
137      MOV      r4, #0
```

Gestion du score joueur 1:

Si le bouton SW1 est pressé (signal bas), le score du joueur 1 augmente.

Si le score atteint 6, il revient à 0 (comme une boucle de score).

Cette logique simule l'incréméntation des points dans Pong.



Affichage du score avec LEDs

```
159      MOV      r0, r4
160      BL      Blink_LED1
161      MOV      r0, r5
162      BL      Blink_LED2
```

Indication du score via LEDs

La LED clignote autant de fois que le score du joueur.

LED PF4 → Joueur 1

LED PF5 → Joueur 2

```
196 BothHit_180
197      BL      MOTEUR_GAUCHE_ARRIERE
198      BL      MOTEUR_DROIT_AVANT
199      LDR      r0, =DELAY_SUBUNIT
200      MOV      r1, #TURN_180_UNITS
201      MUL      r4, r0, r1
202 Delay180
203      SUBS     r4, r4, #1
204      BNE      Delay180
```

Collision frontale — demi-tour

Si les deux bumpers détectent un choc presque simultanément

Le robot exécute un peu plus que un **180°**

Simulation du rebond sur bord frontal dans Pong

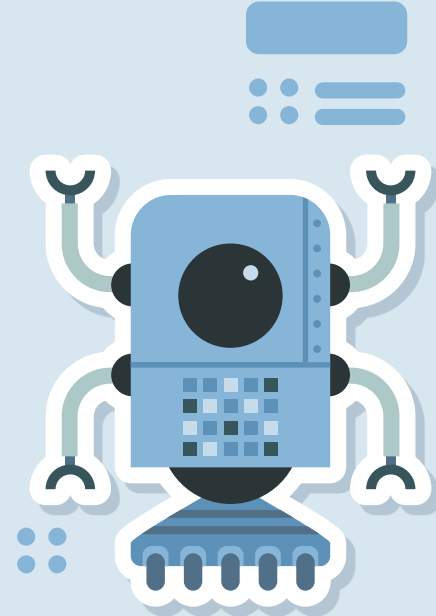
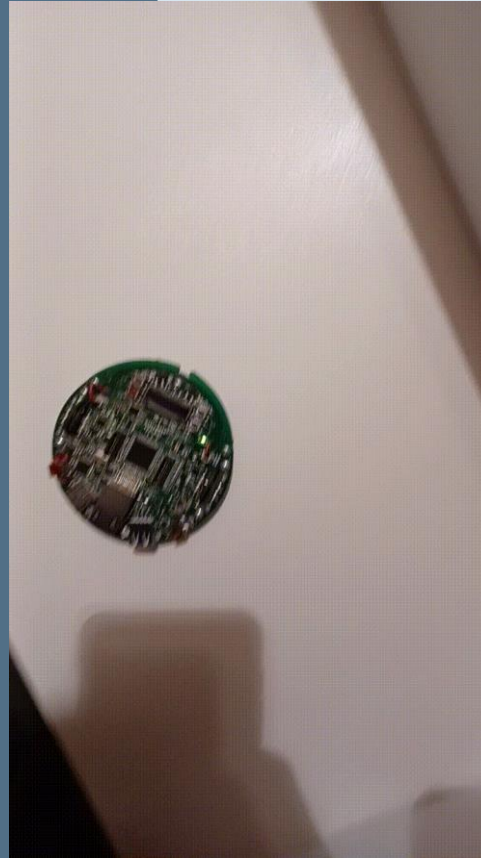
Scénario 1 — Le robot avance sans obstacle

Description :

Le robot se déplace droit en avant, les deux moteurs tournent vers l'avant.

Lien avec Pong :

La balle avance sur le terrain sans toucher de mur ni de raquette.



Scénario 2 — Collision côté gauche (bumper gauche)

Condition : le bumper gauche touche un mur/obstacle.

Action robot :

roue gauche en arrière

roue droite en avant

rotation vers la **droite**

durée aléatoire → angle variable (comme rebond aléatoire)

Lien avec Pong :

La balle touche une paroi et rebondit vers l'autre côté → angle non fixe pour dynamisme.



Scénario 3 — Collision côté droit (bumper droit)

Condition : le bumper droit touche un mur/obstacle.

Action robot :

roue droite en arrière

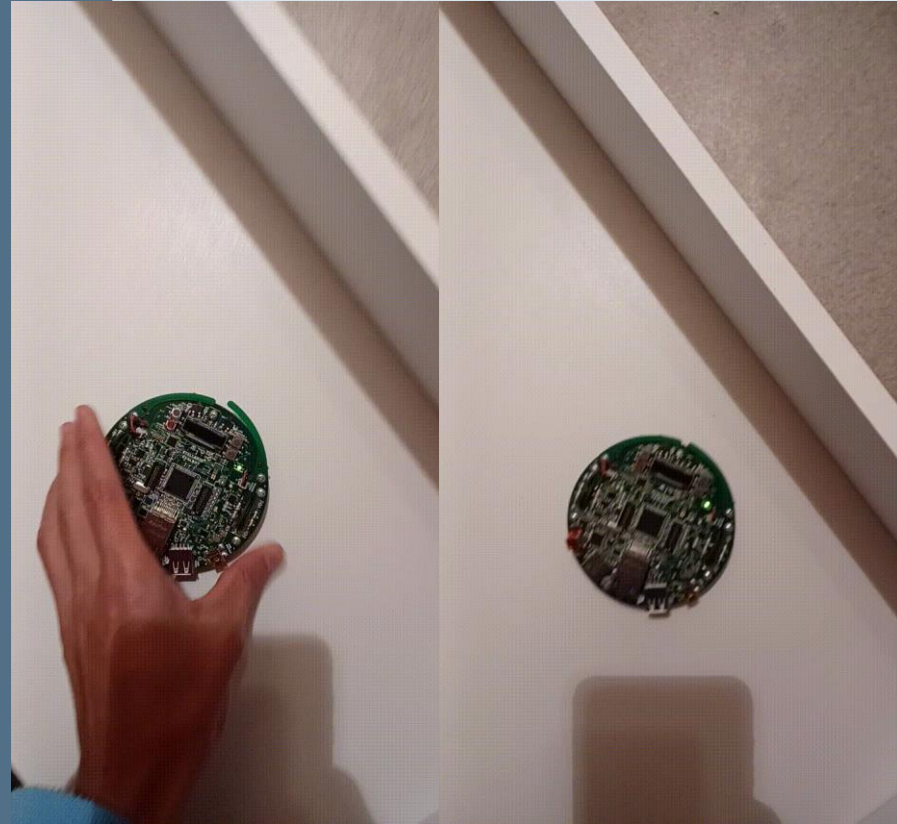
roue gauche en avant

rotation vers la **gauche**

durée aléatoire

Lien avec Pong :

Rebond de la balle dans l'autre sens.



Scénario 4 — Collision frontale (deux bumpers touchés)

Condition : les deux bumpers touchent un obstacle dans un court délai (.

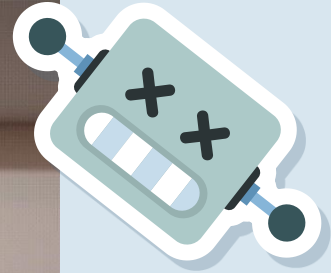
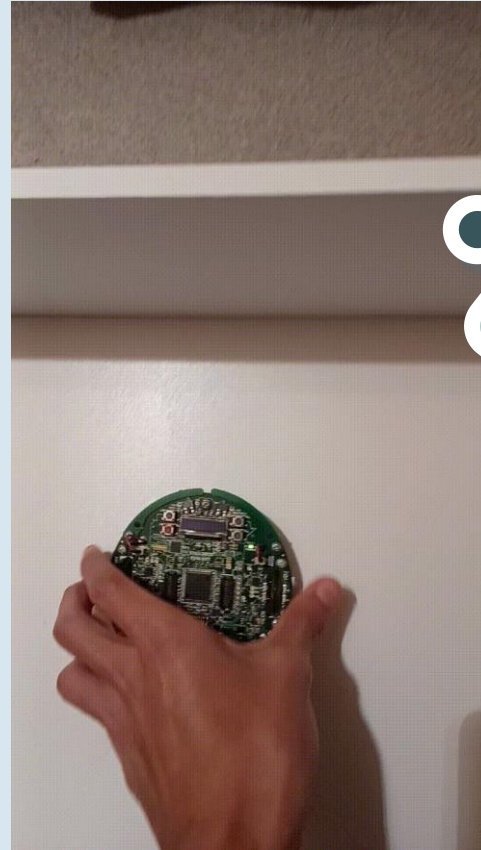
Action robot :

demi-tour (rotation d'environ 200°)

durée **fixe** (pas aléatoire)

Lien avec Pong :

La balle touche une raquette/ligne frontale et repart en sens inverse presque net.



Scénario 5 — Joueur marque un point

Action utilisateur : appui sur SW

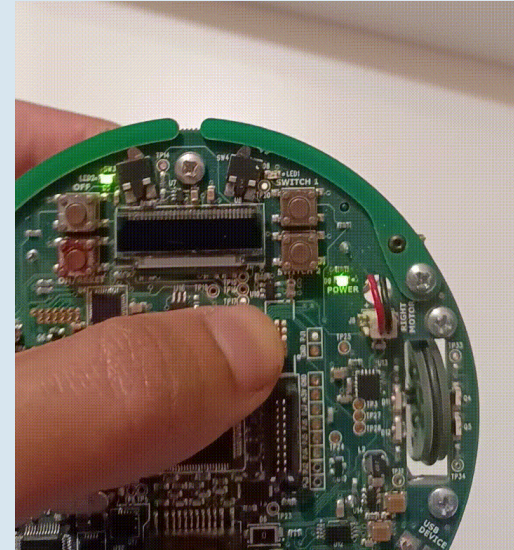
Action robot / système :

Score joueur 1 +1

LED clignote selon le score

Lien avec Pong :

Le joueur marque un point.



**Scénario 6 — Score atteint 5 →
retour à 0**

Condition : si un joueur dépasse le
score max (5)

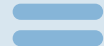
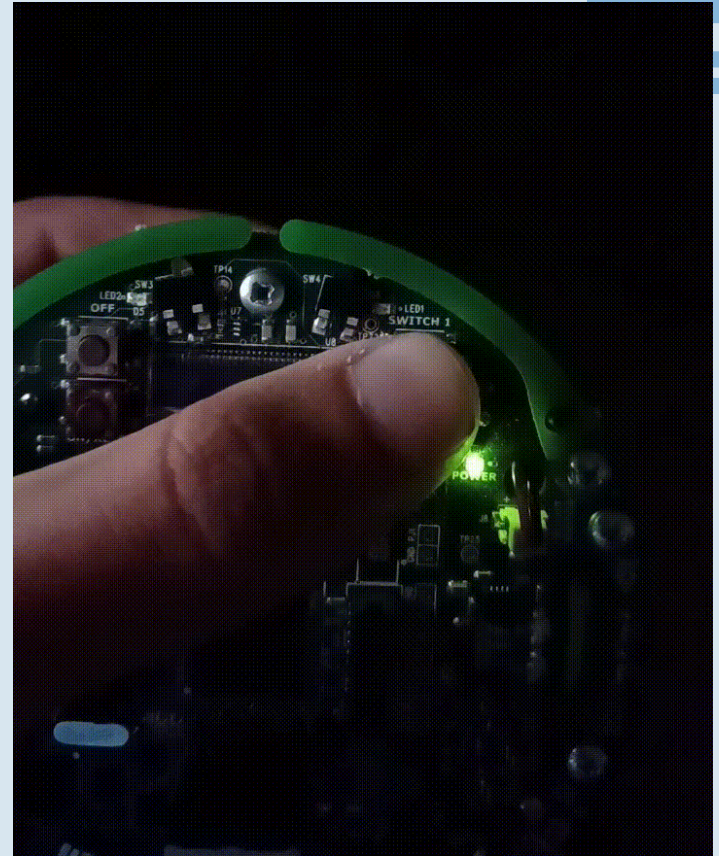
Action robot / système :

Score revient à 0 (cycle)



Lien avec Pong :

Score reset comme un round
terminé.



Choix de configuration des GPIO

LEDs (PF4, PF5) — Sorties pour afficher le score

```
80      LDR      r6, =GPIO_PORTF_BASE + GPIO_O_DIR
81      MOV      r0, #BROCHES_LEDS
82      STR      r0, [r6]
83      LDR      r6, =GPIO_PORTF_BASE + GPIO_O_DEN
84      STR      r0, [r6]
```

LEDs configurées en **sortie numérique** pour clignoter selon le score

Boutons (PD6, PD7) — Entrées avec Pull-Up

```
87      LDR      r6, =GPIO_PORTD_BASE + GPIO_O_PUR
88      MOV      r0, #(BROCHE_SW1 + BROCHE_SW2)
89      STR      r0, [r6]
90      LDR      r6, =GPIO_PORTD_BASE + GPIO_O_DEN
91      STR      r0, [r6]
```

Pull-up interne = bouton appuyé → **niveau bas**.

Accès masqué aux GPIO

```
109     LDR      r8, =GPIO_PORTE_BASE + (BROCHES_BUMPERS << 2)
175     LDR      r1, [r8]
```

On lit **uniquement** les bits nécessaires → sécurisé, pas d'effet sur le reste du port.

Bumpers (PE0, PE1) — Entrées avec Pull-Up

```
102     LDR      r6, =GPIO_PORTE_BASE + GPIO_O_PUR
103     MOV      r0, #BROCHES_BUMPERS
104     STR      r0, [r6]
105     LDR      r6, =GPIO_PORTE_BASE + GPIO_O_DEN
106     STR      r0, [r6]
```

Les bumpers détectent un choc (niveau bas) → rebond du robot.

Pilotage moteurs (fonctions)

```
121     BL       MOTEUR_GAUCHE_AVANT
122     BL       MOTEUR_DROIT_AVANT
```

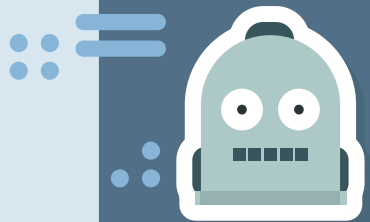
Le robot avance comme la balle de Pong se déplace.

Utilisation du SysTick (Timers & pseudo-random)

```
265     LDR      r0, =0xE000E018
266     LDR      r1, [r0]
```

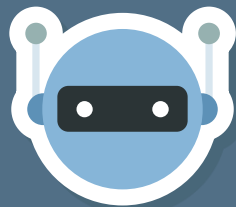
Sert à créer un timing + durée aléatoire de rotation → rebond pas toujours identique.

Problèmes rencontrés



Gestion du rebond et angles

Difficulté à obtenir un rebond cohérent au début
Ajustement du délai et ajout d'un pseudo-aléatoire nécessaire



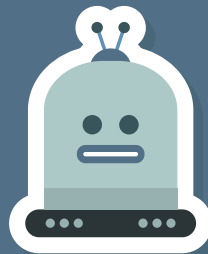
Blocage dû aux délais

Les boucles de delay bloquent la CPU
Pendant un clignotement, le robot ne peut rien faire
Contradiction avec la logique "temps réel" d'un Pong



Détection des bumpers (active-bas)

Les bumpers renvoient **0** quand **activés** → logique inversée*
Il a fallu vérifier l'état avec BEQ et non BNE



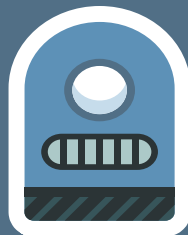
Risque de double-choc

Si le robot touche l'angle d'un obstacle → 2 bumpers activés
Gestion nécessaire pour éviter un comportement incohérent



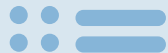
Anti-rebond des boutons

Un appui faisait parfois +1 point plusieurs fois
Nécessité d'ajouter un délai anti-rebond

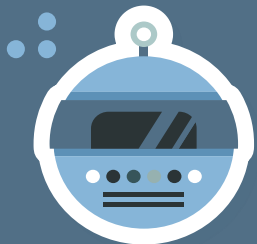


Lisibilité du score avec LEDs

Difficile de compter les clignotements rapidement
D'où l'idée d'ajouter un **OLED** dans les améliorations

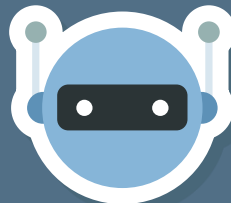


Améliorations possibles



Affichage du score sur écran OLED

Remplacer les LEDs par un écran OLED
Score affiché comme dans le vrai Pong



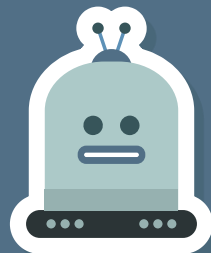
Détection automatique du score

Capteurs (IR / ultrason / caméra) pour détecter quand le robot franchit une zone



Remplacer les delays par interruptions

Robot réactif **en temps réel**
Plus de blocage pendant les clignotements LEDs



Effets sonores

Ajouter buzzer → *bip* au rebond, *pong* au point
Immersion arcade



À vos
questions

